Shared variable (1/7)

```
LONG g_lTotalItems = 0;

unsigned ThreadOneProc( LPVOID lpvUnused) {
...
// thread one has finished processing an item…
g_lTotalItems = g_lTotalItems + 1;
...
}
unsigned ThreadTwoProc( LPVOID lpvUnused) {
...
// thread two has finished processing an item…
g_lTotalItems = g_lTotalItems + 1;
...
}
```

Cohen & Woodring, *Win32 Multithreaded Programming*, p. 65

Copyright © 2001, Viktor Khlebnikov v 0.1

Shared variable (2/7)

Pseudo-assembly language:

```
load   RX, [g_lTotalItems]    ; load value of
                              ; g_lTotalItems
                              ; from memory into
                              ; register RX
inc    RX                     ; increment value
                              ; in RX register
store  [g_lTotalItems], RX    ; store the value
                              ; of the RX register
                              ; into memory
```

Cohen & Woodring, *Win32 Multithreaded Programming*, p. 66

Copyright © 2001, Viktor Khlebnikov v 0.1

Shared variable (3/7)

```
#include <windows.h>
#include <stdio.h>

#define NUM_INCR    10000000L
#define NUM_THRDS   10

// Prototype for Incr application worker thread procedure.
//
DWORD WINAPI ChildThreadProc( LPVOID lpThreadParameter );

LONG  g_lTotal = 0L;
int   StartFlag = NUM_THRDS;

int main( void )
{
  HANDLE  hChildThread[NUM_THRDS];
  DWORD   dwChildThreadId[NUM_THRDS];
  DWORD   dwPrimaryThreadId;

  dwPrimaryThreadId = GetCurrentThreadId();
```

Copyright © 2001, Viktor Khlebnikov v 0.1

Shared variable (4/7)

```
for(int n=0; n<NUM_THRDS; n++) {
  hChildThread[n] =
    CreateThread(
      NULL,
      0,
      ChildThreadProc,
      (LPVOID)n,
      CREATE_SUSPENDED,
      &dwChildThreadId[n]
    );

  if( hChildThread[n] == NULL )
    printf("Failed to create worker thread: error 0x%x\n",
           GetLastError() );

  SetThreadPriority(hChildThread, THREAD_PRIORITY_IDLE);

  printf(
    "[%08x] Child thread %d with ID = 0x%08x has been created.\n",
    dwPrimaryThreadId, n, dwChildThreadId[n] );
}
```

Copyright © 2001, Viktor Khlebnikov v 0.1

Shared variable (5/7)

```
  for(n=0; n<NUM_THRDS; n++) {
    ResumeThread(hChildThread[n]);
    printf(
     "[%08x] Child thread %d with ID = 0x%08x has been activated.\n",
     dwPrimaryThreadId, n, dwChildThreadId[n] );
  }

  WaitForMultipleObjects(NUM_THRDS, hChildThread, TRUE, INFINITE);

  for (n=0; n<NUM_THRDS; n++)
    CloseHandle(hChildThread[n]);

  printf(
    "[%08x] Primary thread has finished with Total = %ld.\n",
    dwPrimaryThreadId, g_lTotal );

  printf(
    "[%08x] Primary thread:          checked Total = %ld.\n",
    dwPrimaryThreadId, NUM_INCR*NUM_THRDS );
  return(0);
}
```

Copyright © 2001, Viktor Khlebnikov v 0.1

Shared variable (6/7)

```
// Thread entry procedure for the worker thread
//
DWORD WINAPI ChildThreadProc( LPVOID lpThreadParameter )
{
  DWORD dwThreadId = GetCurrentThreadId();

  StartFlag--;
  while (StartFlag);

  for (int i=0; i<NUM_INCR; i++)
    g_lTotal++;
//            InterlockedIncrement(&g_lTotal);

  printf(
    "[%08x] Child thread %d has finished.\n",
    dwThreadId, lpThreadParameter );

  return(0);
}
```

Copyright © 2001, Viktor Khlebnikov v 0.1

Shared variable (7/7)

```
[fff697cb] Child thread 0 with ID = 0xfff69dc3 has been created.
[fff697cb] Child thread 1 with ID = 0xfff69edf has been created.
[fff697cb] Child thread 0 with ID = 0xfff69dc3 has been activated.
[fff697cb] Child thread 1 with ID = 0xfff69edf has been activated.
[fff69edf] Child thread 1 has finished.
[fff69dc3] Child thread 0 has finished.
[fff697cb] Primary thread has finished with Total =  19301448.
[fff697cb] Primary thread:          checked Total =  20000000.
```

```
[fff6913f] Child thread 0 with ID = 0xfff69f37 has been created.
[fff6913f] Child thread 1 with ID = 0xfff699c3 has been created.
[fff6913f] Child thread 0 with ID = 0xfff69f37 has been activated.
[fff6913f] Child thread 1 with ID = 0xfff699c3 has been activated.
[fff69f37] Child thread 0 has finished.
[fff699c3] Child thread 1 has finished.
[fff6913f] Primary thread has finished with Total = 17910715.
[fff6913f] Primary thread:          checked Total = 20000000.
```

Copyright © 2001, Viktor Khlebnikov v 0.1

Shared variable: The Interlocked Functions (Win32 API)

To increment/decrement a LONG atomically:

```
LONG InterlockedIncrement( LPLONG lplValue );
LONG InterlockedDecrement( LPLONG lplValue );
```

Parameter	Type	Description
lplValue	LONG *	Pointer to a 32-bit LONG variable that will be incremented/decremented by 1.

To read the current value and replace it with another value atomically:

```
LONG InterlockedExchange(LPLONG lplTarget, LONG lValue);
```

Parameter	Type	Description
lplTarget	LONG *	Pointer to a 32-bit LONG variable that will be changed.
lValue	LONG	New 32-bit value to store into *lplTarget*.

Cohen & Woodring, *Win32 Multithreaded Programming*, p. 67

Copyright © 2001, Viktor Khlebnikov v 0.1